Jenkins Final Assignment

End-to-End CI/CD Pipeline with Docker Agents

Corporate DevOps Training Assignment

Objective

This assignment is designed to help trainees practice end-to-end Jenkins setup and pipeline execution using Docker-based agents. It ensures hands-on experience with Jenkins installation, GitHub integration, environment variables, credential management, Docker image creation, pushing images to DockerHub, and deploying containers.

Step 1 – Jenkins Installation

1. Install Jenkins on a Linux VM (Ubuntu preferred) or run via Docker. 2. Configure Jenkins admin user and install plugins: Docker, Docker Pipeline, Git, Credentials Binding. 3. Verify Jenkins setup with a test job.

Step 2 – GitHub Repository Setup

- Fork and use the repository (Devopsfarm-provided) containing a sample Python Flask app. - The repository must include: • app.py – basic Flask app • requirements.txt – dependencies • tests/test_app.py – pytest test case • Dockerfile – to containerize the app

Step 3 – DockerHub Setup

1. Create a DockerHub account and repository: `username/simple-python-app`. 2. Save credentials in Jenkins with ID: `dockerhub-creds`.

Step 4 – Jenkins Pipeline

Create a Jenkinsfile in the repository with the following key stages: - Checkout: Pull code from GitHub. - Install dependencies: Install packages from requirements.txt. - Run Tests: Execute pytest cases. - Build Docker Image: Build image tagged with Jenkins build number. - Push to DockerHub: Use Jenkins credentials to push the image. - Deploy Container: Run the container locally exposing the app.

Sample Jenkinsfile

```
pipeline { agent { docker { image 'python:3.10' args '-u root:root'
} } environment { IMAGE_NAME = "simple-python-app" DOCKERHUB_REPO
= "your-dockerhub-username/simple-python-app" } stages {
stage('Checkout') { steps { git branch: 'main', url:
'https://github.com/yourusername/simple-python-app.git' } }
stage('Install dependencies') { steps { sh 'pip install -r
requirements.txt' } } stage('Run Tests') { steps { sh 'pytest
--maxfail=1 --disable-warnings -q' } } stage('Build Docker Image')
{ steps { sh "docker build -t $DOCKERHUB_REPO:${BUILD_NUMBER} ." }
```

```
} stage('Push to DockerHub') { steps {
withCredentials([usernamePassword(credentialsId:
'dockerhub-creds', usernameVariable: 'DOCKER_USER',
passwordVariable: 'DOCKER_PASS')]) { sh ''' echo $DOCKER_PASS |
docker login -u $DOCKER_USER --password-stdin docker push
$DOCKERHUB_REPO:${BUILD_NUMBER} ''' } } } stage('Deploy
Container') { steps { sh "docker run -d -p 5000:5000 --name
simpleapp_${BUILD_NUMBER} $DOCKERHUB_REPO:${BUILD_NUMBER}" } } }
post { always { sh "docker ps -a" } success { echo "Build
Successful!" } failure { echo "Build Failed!" } } }
```

Deliverables

- Screenshot of Jenkins dashboard and installed plugins. - Screenshot of pipeline run with all stages green. - Screenshot of DockerHub repo with pushed image. - Screenshot of app running at http://localhost:5000. - GitHub repo link with Jenkinsfile and app code.

Bonus Tasks

- Add parallel stages (Python 3.8 & 3.10 tests). - Configure Slack/Email notifications. - Use docker-compose for multi-service deployment.

Devopsfarm IT Solutions

Contact: Omprakash Paliwal – 9971566583